vectors and data frames in R

prof. dr. nicolas meseth

1. from a value to a table
2. vectors
3. data frames

a type case keeps many small things in order because every column holds only one kind of thing.



image: ai-generated (gpt-image-2.5-flare)

part 1

from a value to a table

four shapes of data

2

in a matrix every cell has the same type, in a data frame every column has its own.



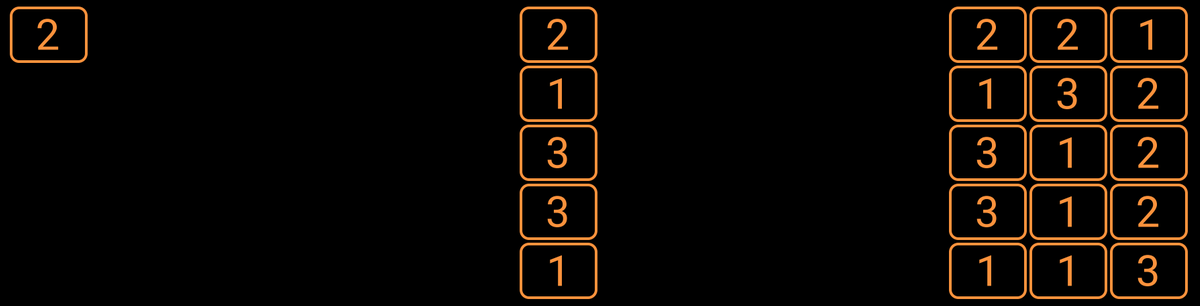


scalar value

vector

in a matrix every cell has the same type, in a data frame every column has its own.

four shapes of data



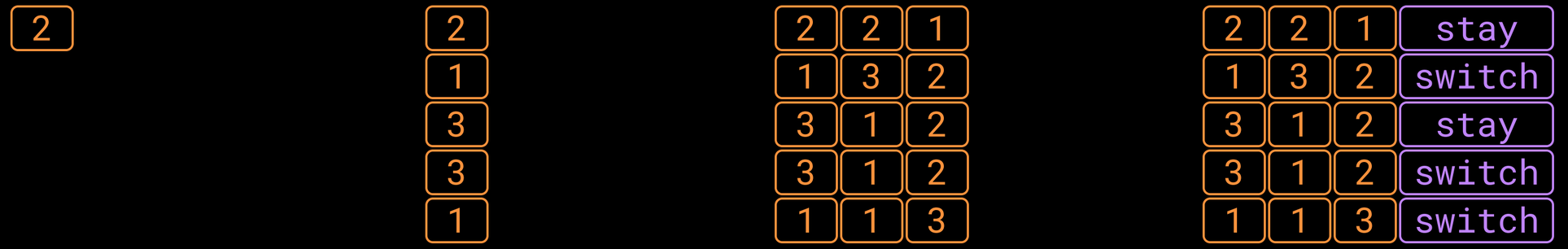
scalar value

vector

matrix

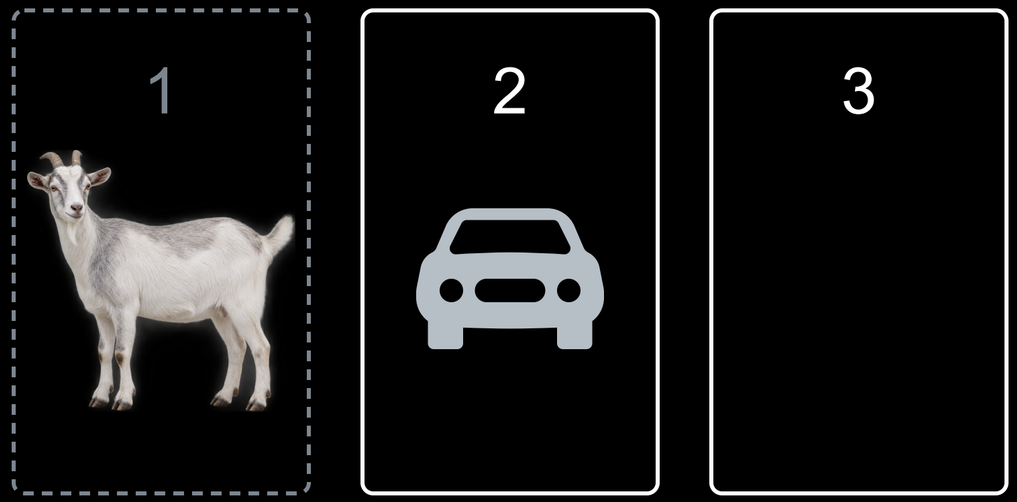
in a matrix every cell has the same type, in a data frame every column has its own.

four shapes of data



scalar value | vector | matrix | data frame

in a matrix every cell has the same type, in a data frame every column has its own.



opened by the host your pick

```
prize_door          2
contestant_choice   2
monty_opens         1
decision            "stay"
```

a game show with three doors: you pick one, the host opens another with a goat behind it, and you stay with your door or switch.

image: ai-generated (gpt-image-2.5-flare)

play the game on the site, and stay or switch.

after a few rounds the game shows your rounds as R code, one line per fact.

hands-on-computer-science.de/ds/en/items/monty-hall-game



```
prize_door <- c(2, 1, 3, 3, 1)
contestant_choice <- c(2, 3, 1, 1, 1)
monty_opens <- c(1, 2, 2, 2, 3)
decision <- c("stay", "switch", "stay", "switch", "switch")
```

c() combines values into a vector, and the first value of each vector belongs to the first round.

part 2

vectors

many values of one kind

the same position, the same round

	1	2	3	4	5
prize_door	2	1	3	3	1
contestant_choice	2	3	1	1	1
monty_opens	1	2	2	2	3
decision	stay	switch	stay	switch	switch

four vectors of the same length describe five rounds, as long as nobody mixes up the order.

the same position, the same round

	1	2	3	4	5
prize_door	2	1	3	3	1
contestant_choice	2	3	1	1	1
monty_opens	1	2	2	2	3
decision	stay	switch	stay	switch	switch

round 1: car behind door 2, door 2 picked, host opened door 1, player stayed

four vectors of the same length describe five rounds, as long as nobody mixes up the order.

the same position, the same round

	1	2	3	4	5
prize_door	2	1	3	3	1
contestant_choice	2	3	1	1	1
monty_opens	1	2	2	2	3
decision	stay	switch	stay	switch	switch

round 4: car behind door 3, door 1 picked, host opened door 2, player switched

four vectors of the same length describe five rounds, as long as nobody mixes up the order.

one vector, one type

you write	R holds	typeof()
`c(2, 1, 3)`	2 1 3	`"double"`
`c(2L, 1L, 3L)`	2 1 3	`"integer"`
`c("stay", "switch")`	"stay" "switch"	`"character"`
`c(TRUE, FALSE)`	TRUE FALSE	`"logical"`

every value in a vector has the same type, and typeof() tells you which one.

```
length(prize_door)     # 5
typeof(decision)       # "character"
mean(prize_door)       # 2
table(decision)        # stay 2, switch 3
```

sum, mean, median, min, max, range, sd, sort, unique, table: functions that take a whole vector at once.

```
c(1, TRUE)
```




```
c(1, TRUE, "stay")
```



R never mixes types in a vector, it quietly converts to the most general one.

```
c(1, TRUE)
```




```
"double"
```

```
c(1, TRUE, "stay")
```



R never mixes types in a vector, it quietly converts to the most general one.

`c(1, TRUE)` → `1` `1` → `"double"`

`c(1, TRUE, "stay")` → `"1"` `"TRUE"` `"stay"` → `"character"`



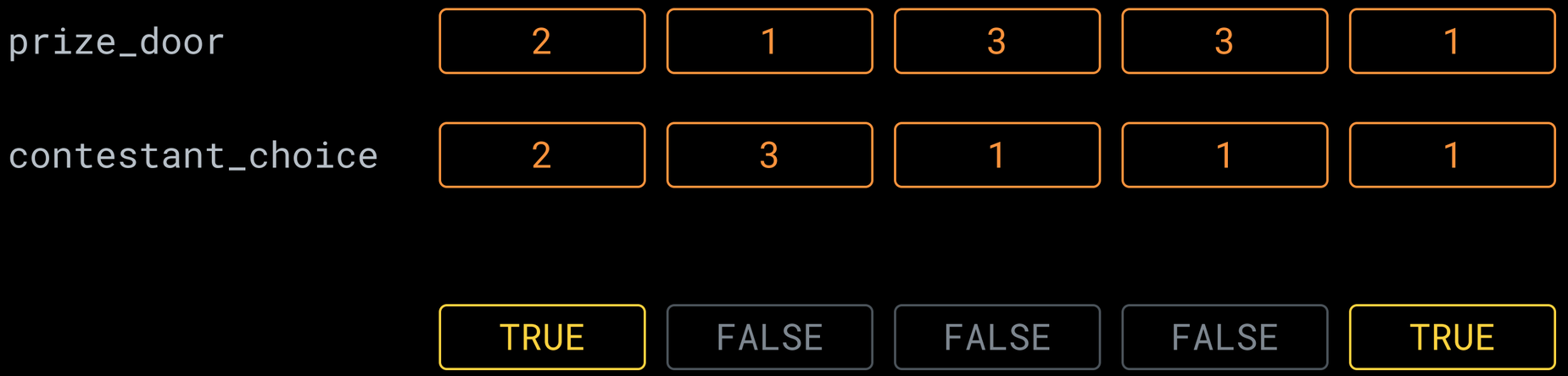
R never mixes types in a vector, it quietly converts to the most general one.

comparing whole vectors



an operation on two vectors works position by position, with no loop to write.

comparing whole vectors



an operation on two vectors works position by position, with no loop to write.

```
first_pick_right <- prize_door == contestant_choice
sum(first_pick_right)      # 2
mean(first_pick_right)     # 0.4
```

R counts TRUE as 1 and FALSE as 0, so the mean of a logical vector is a share.

picking values by position

```
decision[1]          # "stay"
```

square brackets select, a minus leaves out, and the result is again a vector.

```
decision[1]          # "stay"
decision[-1]         # all but the first
```

square brackets select, a minus leaves out, and the result is again a vector.

```
decision[1]          # "stay"
decision[-1]         # all but the first
decision[2:4]        # "switch" "stay" "switch"
```

square brackets select, a minus leaves out, and the result is again a vector.

picking values by position

```
decision[1]          # "stay"
decision[-1]         # all but the first
decision[2:4]        # "switch" "stay" "switch"
decision[c(1, 3)]    # "stay" "stay"
```

square brackets select, a minus leaves out, and the result is again a vector.

picking values by a condition

```
decision == "stay"
```

TRUE	FALSE	TRUE	FALSE	FALSE

```
first_pick_right
```

TRUE	FALSE	FALSE	FALSE	TRUE

a logical vector in square brackets keeps exactly the positions that are TRUE.

```
decision == "stay"
```

TRUE	FALSE	TRUE	FALSE	FALSE

```
first_pick_right
```

TRUE	FALSE

```
first_pick_right[decision == "stay"]
```

a logical vector in square brackets keeps exactly the positions that are TRUE.

```
x <- c(2, NA, 3)
```




```
mean(x)                  → NA
mean(x, na.rm = TRUE)    → 2.5
```

related, but not the same

`NULL`	nothing at all
`NaN`	0 / 0, not a number
`Inf`	1 / 0, infinite

NA means "not known", so anything computed with it is not known either, until you say what to do.

```
decision_factor <- factor(
  decision,
  levels = c("stay", "switch")
)
levels(decision_factor)    # "stay" "switch"
```

a factor knows all its possible values and their order, even those that do not occur.

a vector is a row of values of one type, and position `i` belongs to observation `i`.

part 3

data frames

vectors side by side

vectors become a table

```
games <- data.frame(prize_door, contestant_choice, monty_opens, decision)
```

prize_door	contestant_choice	monty_opens	decision
2	2	1	stay
1	3	2	switch
3	1	2	stay
3	1	2	switch
1	1	3	switch

a data frame is a list of equally long vectors, and every vector holds values of one type.

vectors become a table

```
games <- data.frame(prize_door, contestant_choice, monty_opens, decision)
```

	prize_door	contestant_choice	monty_opens	decision
1	2	2	1	stay
2	1	3	2	switch
3	3	1	2	stay
4	3	1	2	switch
5	1	1	3	switch

a data frame is a list of equally long vectors, and every vector holds values of one type.

vectors become a table

```
games <- data.frame(prize_door, contestant_choice, monty_opens, decision)
```

	prize_door	contestant_choice	monty_opens	decision
1	2	2	1	stay
2	1	3	2	switch
3	3	1	2	stay
4	3	1	2	switch
5	1	1	3	switch

one column
is one vector

one row
is one game

a data frame is a list of equally long vectors, and every vector holds values of one type.

```
monty_hall <- read.csv("lets_make_a_deal.csv")
str(monty_hall)
```

```
'data.frame':  100 obs. of  5 variables:
 $ contestant       : chr  "Charles" "Patricia" "Karen" "Jessica" ...
 $ prize_door       : int  3 1 3 3 1 1 3 2 3 3 ...
 $ contestant_choice: int  3 3 3 1 3 3 1 3 1 2 ...
 $ monty_opens      : int  2 2 1 2 2 2 2 1 2 1 ...
 $ decision         : chr  "stay" "stay" "stay" "stay" ...
```

str() shows every column with its type and its first values, and it is the first thing to run on new data.

a vector or a table?

contestant	prize_door	contestant_choice	monty_opens	decision
Charles	3	3	2	stay
Patricia	1	3	2	stay
Karen	3	3	1	stay
Jessica	3	1	2	stay
Joseph	1	3	2	stay
Robert	1	3	2	stay

$ takes the vector out of the table, single brackets keep a smaller table.

a vector or a table?

contestant	prize_door	contestant_choice	monty_opens	decision
Charles	3	3	2	stay
Patricia	1	3	2	stay
Karen	3	3	1	stay
Jessica	3	1	2	stay
Joseph	1	3	2	stay
Robert	1	3	2	stay

```
monty_hall$decision
```

```
[1] "stay" "stay" "stay" "stay" "stay" "stay"
```

$ takes the vector out of the table, single brackets keep a smaller table.

a vector or a table?

contestant	prize_door	contestant_choice	monty_opens	decision
Charles	3	3	2	stay
Patricia	1	3	2	stay
Karen	3	3	1	stay
Jessica	3	1	2	stay
Joseph	1	3	2	stay
Robert	1	3	2	stay

decision
stay
stay
stay
stay
stay
stay

`monty_hall$decision`

`monty_hall["decision"]`

`stay` `stay` `stay` `stay` `stay` `stay`

$ takes the vector out of the table, single brackets keep a smaller table.

```
monty_hall[1, ]      # the first row
```

[rows, columns]: an empty place means all of them.

```
monty_hall[1, ]      # the first row
monty_hall[, 2]      # the second column, a vector
```

[rows, columns]: an empty place means all of them.

```
monty_hall[1, ]      # the first row
monty_hall[, 2]      # the second column, a vector
monty_hall[1:10, c("prize_door", "decision")]
```

[rows, columns]: an empty place means all of them.

```
monty_hall[1, ]      # the first row
monty_hall[, 2]      # the second column, a vector
monty_hall[1:10, c("prize_door", "decision")]
nrow(monty_hall)      # 100
ncol(monty_hall)      # 5
```

[rows, columns]: an empty place means all of them.

a new column from the others

```
monty_hall$won <-
  (monty_hall$decision == "stay" &
     monty_hall$prize_door == monty_hall$contestant_choice) |
  (monty_hall$decision == "switch" &
     monty_hall$prize_door != monty_hall$contestant_choice)
```

prize_door	contestant_choice	decision	won
3	3	stay	TRUE
1	3	stay	FALSE
3	3	stay	TRUE
3	1	stay	FALSE
1	3	stay	FALSE
1	3	stay	FALSE

a column is a vector,
so everything from part 2
works on it.

monty_hall

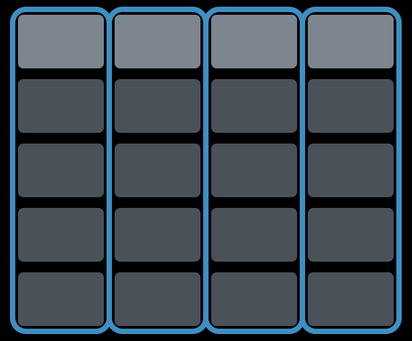
```
86       Linda          2                1          3     stay FALSE
87      Robert          3                3          2     stay  TRUE
88     Barbara          1                3          2     stay FALSE
89       Karen          2                2          1     stay  TRUE
90       Sarah          1                3          2   switch  TRUE
91      Joseph          1                1          3     stay  TRUE
92     Richard          1                2          3   switch  TRUE
93     Charles          1                1          3     stay  TRUE
94       Susan          3                1          2     stay FALSE
95    Patricia          1                3          2     stay FALSE
96     Richard          1                2          3     stay FALSE
97    Patricia          1                3          2   switch  TRUE
98       Susan          3                3          1     stay  TRUE
99     Jessica          1                2          3     stay FALSE
100   Patricia          1                1          3     stay  TRUE
```

as_tibble(monty_hall)

```
# A tibble: 100 × 6
   contestant prize_door contestant_choice monty_opens decision won
   <chr>           <int>              <int>       <int> <chr>    <lgl>
 1 Charles             3                  3           2 stay     TRUE
 2 Patricia            1                  3           2 stay     FALSE
 3 Karen               3                  3           1 stay     TRUE
 4 Jessica             3                  1           2 stay     FALSE
 5 Joseph              1                  3           2 stay     FALSE
 6 Robert              1                  3           2 stay     FALSE
 7 John                3                  1           2 stay     FALSE
 8 Richard             2                  3           1 switch   TRUE
 9 Michael             3                  1           2 stay     FALSE
10 Elizabeth           3                  2           1 switch   TRUE
# ℹ 90 more rows
```

a tibble is a data frame with better manners: it shows the types and does not flood the console.

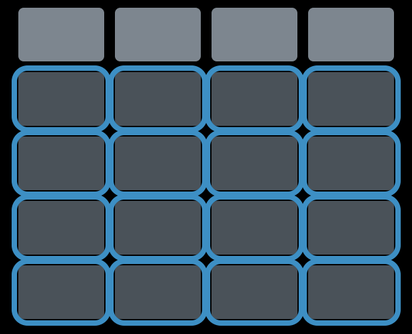
tidy data



each variable is a column



each observation is a row



each value is a cell

when a table is tidy, every column is a vector of one variable, and the tools of R fit it.

after Wickham, Çetinkaya-Rundel & Grolemund: R for Data Science, chapter "data tidying"

a data frame is a list of equally long vectors, and every vector holds values of one type.

to read on: Hands-On Programming with R, chapter "R objects", and R for Data Science, chapters "data tidying", "logical vectors", "factors" and "missing values"